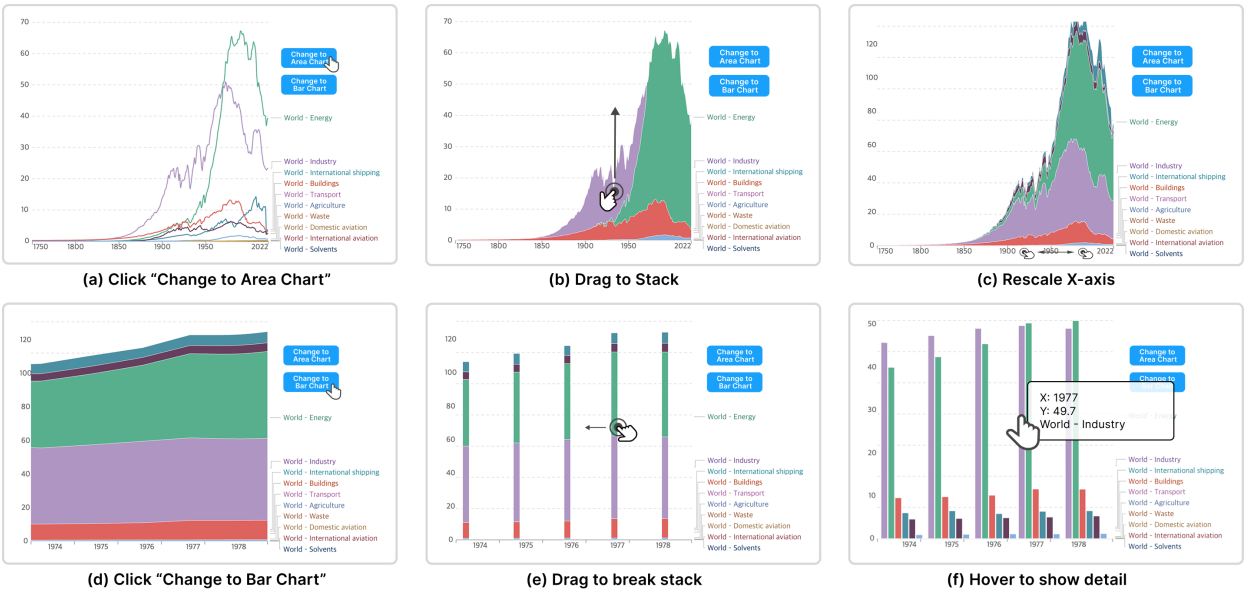
Athanor: Authoring Action Modification-based Interactions on Static Visualizations via Natural Language

Can Liu, Jaeuk Lee, Tianhe Chen, Zhibang Jiang, Xiaolin Wen and Yong Wang



(a) Click "Change to Area Chart" (b) Drag to Stack (c) Rescale X-axis

(d) Click "Change to Bar Chart" (e) Drag to break stack (f) Hover to show detail

Figure 1: Given a static chart (a), *Athanor* supports enabling interactions for it. The static chart has been enhanced with buttons to modify chart types (transforming (a) to (b) and (d) to (e)), the ability to rearrange the visual elements to stack or group ((b) to (c) and (e) to (f)), rescaling on the x-axis ((c) to (d)), and hover functionality to display a detailed tooltip (f). After authoring, the static line chart can support interactive features that enable users to perform in-depth analysis.

Abstract—Interactivity is crucial for effective data visualizations. However, it is often challenging to implement interactions for existing static visualizations, since the underlying code and data for existing static visualizations are often not available, and it also takes significant time and effort to enable interactions for them even if the original code and data are available. To fill this gap, we propose *Athanor*, a novel approach to transform existing static visualizations into interactive ones using multimodal large language models (MLLMs) and natural language instructions. Our approach introduces three key innovations: (1) an action-modification interaction design space that maps visualization interactions into user actions and corresponding adjustments, (2) a multi-agent requirement analyzer that translates natural language instructions into an actionable operational space, and (3) a visualization abstraction transformer that converts static visualizations into flexible and interactive representations regardless of their underlying implementation. *Athanor* allows users to effortlessly author interactions through natural language instructions, eliminating the need for programming. We conducted two case studies and in-depth interviews with target users to evaluate our approach. The results demonstrate the effectiveness and usability of our approach in allowing users to conveniently enable flexible interactions for static visualizations.

Index Terms—User interfaces, interaction authoring, interactive visualization, multi-agent

1 Introduction

Interactivity adds a new dimension to visualizations, enabling users to dynamically explore data, uncover hidden patterns, and tailor views to their specific needs. For instance, interactive visualization software like Tableau and Power BI enable real-time filtering and zooming, revealing insights that are typically inaccessible in static visualizations. However, despite the power of these interactive visualization tools, static visualizations remain prevalent in published outputs [56], typically presenting data from a fixed perspective with underlying data and code usually inaccessible to viewers. This static nature limits users' ability to adjust focus, compare different time periods, and segment data, thereby hindering users from gaining deeper data insights. Interactivity can significantly enhance the utility of static visualizations, yet integrating it into static formats is far from being straightforward and easy [48]. There are three major challenges that have limited the adoption of interactive visualizations:

- **C1. Interaction Complexity:** The interaction space is vast as it encompasses multidimensional design space ranging from different actions (e.g., clicking, hovering, dragging) to various analytical intents (e.g., filtering, comparing, and hypothesis validation) [13, 59]. It is often time-consuming to enable interactions for a static visualization. Even with access to the original code and data, modifying each visualization's code to add interactive features requires substantial visualization expertise and implementation skills.
- **C2. User Requirement Diversity:** The space of user-required in-

- *C. Liu, T. Chen, X. Wen, and Y. Wang are with Nanyang Technological University. E-mail: can.liu@ntu.edu.sg, {chen1594, xiaolin004}@e.ntu.edu.sg, yong-wang@ntu.edu.sg. Y. Wang is the corresponding author.*
- *J. Lee is with Ajou University. E-mail: woukl22@ajou.ac.kr.*
- *Z. Jiang is with Providence Health & Services. E-mail: zhibang.jiang@newschool.edu.*

Manuscript received xx xxx. 201x; accepted xx xxx. 201x. Date of Publication xx xxx. 201x; date of current version xx xxx. 201x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org.

teraction functionalities is enormous. Different users have distinct requirements based on their specific scenarios, tasks, and analysis goals. For example, data analysts may need complex filtering and comparison interactions, while casual users prefer simple highlighting and annotation features. This diversity in user requirements across different contexts makes it challenging to provide a standardized implementation of visualization interactions.

- **C3. Visualization Implementation Diversity:** There are various types of visualizations (e.g., bar charts, area charts, line charts, etc.) and visualizations can be implemented using different toolkits, e.g., Vega-Lite [40], ECharts [21], and D3 [3], resulting in substantially different code implementations. This heterogeneity makes it difficult to enable interactions for visualizations implemented by different visualization toolkits. Furthermore, many existing visualizations lack access to the underlying data and code, making it infeasible to support interactivity by extending the original implementation.

To address these challenges, we propose *Athanor*, a novel approach that allows users to **add interactions to existing static visualizations conveniently and efficiently** based on users' natural language inputs. Our approach comprises *an action-modification interaction design space*, *a multi-agent requirement analyzer*, and *an implementation-agnostic representation translator*. The **action-modification interaction design space** maps interactions to *user actions* and their corresponding *visualization modifications*, providing a structured framework to describe the potential interactions supported by a static visualization's information. This design space specifies how user actions trigger specific visualization modifications, forming an actionable interaction model (**C1**). The **multi-agent requirement analyzer** interprets users' natural language requirements, transforming them into actionable specifications within the action-modification design space. Through a collaborative agent-based approach, it processes diverse inputs to precise operations by translating inputs into structured sequences of actions and modifications, correcting parsing errors, and providing guidance to refine unclear requests (**C2**). The **visualization abstraction translator** converts existing visualizations into a flexible abstraction independent of the underlying implementation (**C3**). The translator leverages multimodal large language models (MLLMs) to parse existing visualizations, representing the results through a constraint model [27], which is an implementation-agnostic representation for existing visualizations.

Athanor can effectively parse the content of existing visualizations in SVG format, especially the most popular charts (e.g., bar charts, line charts, scatter plots, and area charts), and then directly enable interactivity on them, which can maintain the visual consistency between the interactive visualizations and the original static visualizations. To demonstrate the effectiveness of *Athanor*, we first present two case studies to illustrate how users can conveniently enable interactions for a given visualization with the help of our approach. Also, we conducted in-depth user interviews with 11 participants. The feedback indicates that our action-modification design space can cover various user requirements, and *Athanor* allows users to effectively add interactions to static visualizations with natural language inputs. The contributions of this work are as follows:

- We propose *Athanor* that consists of three major components: action-modification interaction design space, multi-agent requirement analyzer, and visualization abstraction translator. It allows users to conveniently author flexible interactions for static visualizations via natural language interaction.
- We present two case studies and conduct in-depth user interviews to demonstrate the effectiveness and usefulness of *Athanor*. A gallery of interaction-enhanced visualizations created by using our approach can be found at `https://authorvisinter.github.io`.

2 Related Work

Our work provides a natural language interface (NLI) for authoring interactions on static visualizations, which relates to visualization interaction, NLI for visualization, and visualization enhancement.

2.1 Visualization Interaction

Visualization interaction encompasses various ways users engage with and manipulate visual representations of data. We treat a visualization interaction as user actions and visualization modifications.

Action-based interactions. Several researchers have categorized visualization interactions based on user actions performed via input devices such as mice, keyboards, and touchpads. Shneiderman [47] outlines WIMP (windows, icons, menus, pointer) actions to present users' interaction intentions, such as clicking to zoom and typing to filter, emphasizing direct manipulation with mouse and keyboard inputs. Expanding on this, Ward et al. [55] proposed a taxonomy of actions, including pointing (e.g., mouse clicks to select), dragging (e.g., mouse movements to resize or reposition), scrolling (e.g., mouse wheel or touchpad gestures to navigate), and keypressing (e.g., keyboard shortcuts to toggle views). These physical action frameworks have influenced interactive visualization systems. For example, Polaris [49] maps mouse-based actions (e.g., clicking to highlight, dragging to brush, or context-clicking to drill down) to specific responses. Several approaches [2, 39] studied touchpad interactions, identifying multi-touch gestures like pinching to zoom or swiping to pan as actions in visualization systems.

Modification-based interactions. Another perspective on visualization interaction focuses on how visualization modifies its presentation to achieve users' goals. Yi et al. [59] categorized visualization interactions into several types: select (marking something as interesting), reconfigure (showing a different arrangement), encode (giving a different representation), abstract/elaborate (showing more or less detail), filter (showing something conditionally), and connect (showing related items). Brehmer and Munzner [4, 34] classified visualization modifications into six categories: selecting, navigating, arranging, changing, filtering, and aggregating. Sedig and Parsons [42] offered an alternative taxonomy distinguishing between unipolar and bipolar interactions.

In our work, we define an interaction as comprising both user actions and the corresponding visualization modifications. We categorize user actions according to different input devices (e.g., mouse, touchpad, or keyboard) and classify modifications based on the possible operations that can be performed on an existing visualization.

2.2 NL Interface for Visualization

Over the past decades, the field of natural language interfaces (NLIs) for visualization [25, 29, 30, 45, 53, 54] has attracted significant attention.

NLI for constructing visualizations. Cox et al. [8] introduced the first pipeline for constructing visualizations from natural language input. To address the inherent ambiguity of natural language, machine learning (ML)-based methods [50] and interactive-based methods [10] have been proposed. Articulate [50] employs machine learning techniques to classify visualization tasks based on the words in sentences. DataTone [10] introduces ambiguity widgets that facilitate interaction to reduce ambiguity. Shi et al. [46] proposed a method for decomposing high-level questions into simpler components, enabling the generation of visual answers. Previous methods primarily rely on rule-based or learning-based techniques at the word level, but they are limited in handling the large diversity present at the sentence level. In recent years, large language models (LLMs) [57] (e.g., BERT [9], T5 [38], ChatGPT [35]) have demonstrated impressive capabilities for adapting to visualization generation tasks [17, 22]. For example, Maddigan et al. [33] and Luo et al. [31] translate natural language to visualization specificatio

NLI for existing visualizations. While the approaches mentioned above focus on constructing visualizations from data, some methods [14, 16, 18, 20, 43, 51] focused on existing visualizations. Eviza [43] and Evizeon [14] convert natural language input into filters applied to visualizations. Kim et al. [18] generate explanations to answer questions related to existing visualization charts. Some methods aim to generate natural language content for existing visualizations, for example, generate description [26] and title [24]. Instead of producing explanations, Lai et al. [20] emphasize highlighting charts to help users better understand them.

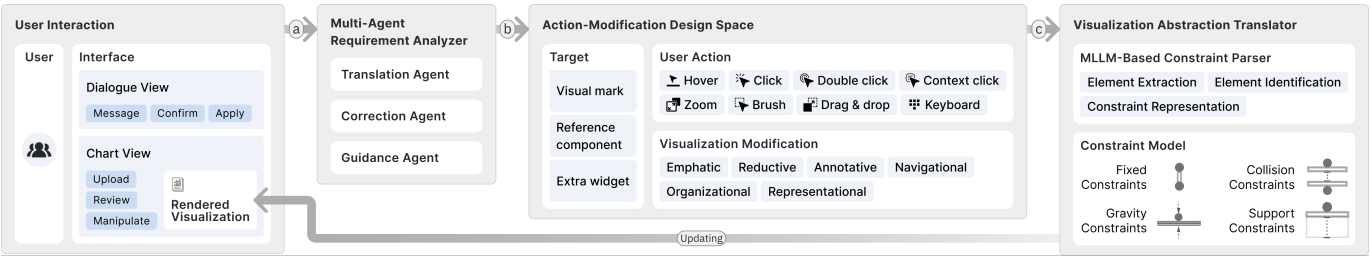


Figure 2: Users can upload existing visualizations in the chart view and present their authoring requirements (a) in the dialogue view. The multi-agent requirement analyzer employs translation, correction, and guidance agents that work collaboratively to translate users' requirements into specifications defined in (b) is the action-modification design space. The uploaded visualizations are abstracted to constraint-based presentations, and the analyzed modifications are also represented through constraints (c). Driven by these constraints, the visualization becomes interactive.

While previous methods mainly directly translate users' natural language input into specifications, recent research focuses on enhancing accuracy through iterative feedback. We propose a multi-agent analyzer that not only translates users' input, but also corrects LLM output and provides guidance for user interaction.

2.3 Visualization Enhancement

Several approaches aim to enhance visualizations. Interaction+ [28] enhanced web visualizations by parsing visual mark attributes and developing an interaction add-on, focusing on non-spatial attributes like color and opacity. Building on D3 specifications, Harper and Agrawala [11, 12] developed tools for deconstructing D3 visualizations, enabling reuse with Vega-lite templates [40]. Beyond D3 visualizations, DataWink [58] employed LLMs to enable the reuse of general SVG visualizations. VisDock [7] provided a system allowing programmers to add interactions (selecting, filtering, navigation) to existing visualizations through code. PapARvis [60] and VisAR [19] focus on enhancing visualization in augmented reality environments. Liu et al. [27] proposed a spatial constraint-based method for modeling the spatial relationships between visualization elements. Similarly, DIVI [48] introduced a framework for adding interactions to existing charts. However, DIVI lacked awareness of spatial layout constraints. For example, if a user deletes a lower segment in a stacked area chart, DIVI cannot automatically shift the remaining elements to maintain visual integrity. MoVer [32] introduced a formal verification language to check spatio-temporal properties of generated animations and feeds errors back to the LLM for correction. In contrast, our work empowers users to define customized interactions according to their specific needs, thereby offering greater flexibility across a diverse range of scenarios and use cases.

3 Overview

Athanor enables users to add interactions to existing visualizations based on natural language requirements. We address three key challenges: structuring interaction possibilities, interpreting user requirements, and supporting diverse visualization implementations. As shown in Figure 2, *Athanor* consists of three parts:

- An **action-modification interaction design space** that abstracts interactions into *user actions* and *visualization modifications* (section 4).
- A **multi-agent requirement analyzer** that analyzes users' natural language instructions into actionable interaction specifications (presented by actions and modifications) (section 5).
- A **visualization abstraction translator** that converts existing visualizations into a flexible representation (section 6), independent of the underlying toolkits or platforms. This representation establishes a foundation for visualization modifications.

4 Action-Modification Design Space

Interaction in a visualization system is a dynamic process where users manipulate data or its representation to explore, analyze, or derive insights [6]. Drawing from foundational works in visualization research [13, 59], we summarize interactions in visualization into two key components: user actions and visualization modifications. A **modification** is defined as the response or outcome that occurs after a user performs an **action**, and a **target** is the object of an action or a modification. We discuss the design space of target, action, and modification in the following subsections.

4.1 Target

A target refers to the object of action and modification. For example, when users hover over a bar, the bar constitutes the target of this action. Targets can be categorized into three types:

- **Visual mark** is the visual element that represents data in visualizations. Visual marks include forms such as **data points, lines, bars, and areas**, which convey information through visual attributes such as **position, size, and color**. For instance, in a bar chart, each **bar** is a visual mark, with its height corresponding to a specific value. Similarly, the lines of a line chart are visual marks.
- **Reference component** refers to visualization components that are not directly bound with data, e.g., coordinate axes and legends. The axes can encode categorical, temporal, and quantitative data attributes. The legend can encode data attributes to color. For example, in a bar chart, the x axis represents categorical labels, while the y axis denotes the quantity of each category.
- **Extra widgets** encompass additional UI elements that facilitate interaction with the visualization, such as **buttons**. For instance, users may want to click an extra button to reorder the bars in a bar chart based on their magnitude.

4.2 User Action

User actions are the means by which users express intentions through input devices (mouse, touchpad, or keyboard) to manipulate targets such as visual marks, reference elements, or widgets. Users often begin by examining visual objects without altering the display. For example, they may *hover* over elements to view details. Once specific data items attract attention, users can select points of interest, either by *clicking* a single element (e.g., a data point in a scatter plot) or *brushing* a group of elements within a region. After selection, users may rearrange elements using *drag and drop*, or adjust the view with *zoom* or *double click* to explore data at different scales. Some actions have fixed semantic meanings, such as pressing "delete" to remove a selected element or performing a context click to access additional options. Building on intentions identified in prior frameworks, our eight-category scheme restructures and clarifies existing taxonomies whose boundaries and action-level mappings remain vague for computational use.

- **Hover** is the action of hovering the cursor over a specific element without clicking. For example, in a line chart, hovering over a data point may display a tooltip with additional information.
- **Click** refers to a single, discrete action performed via an input device (e.g., mouse, touchscreen) to select or activate a specific element in a visualization.
- **Double-click** refers to two rapid consecutive presses or taps on the same element using an input device (e.g., mouse or touchscreen). It is often used to trigger a zooming response in visualizations.
- **Context-click** refers to an access input operation via an input device (e.g., mouse, touch screen) on a specific element, typically to access a context-sensitive menu or additional options. It can be a click using a mouse, a double tap on a touch screen, or a long-press on a touch screen.

- **Zoom** is the action of enlarging or reducing a specific area, often performed using the mouse wheel or pinch gestures on a touchpad. For instance, zooming on the X-axis by scrolling the mouse wheel or pinching on a touchpad can adjust the visible data range, allowing users to focus on specific sections of the dataset.
- **Brush & Drag** refers to the action of selecting a range or region by clicking and dragging the mouse over an area. For example, in a scatter plot, brushing can be used to select a subset of data points for further analysis.
- **Drag & drop** involves clicking on an element, moving the mouse while holding the button, and then releasing it. For example, in a **bar chart**, dragging a bar in a stacked bar chart can change the order of the stacked elements.
- **Keyboard** is the action of pressing the keys. For example, users can press the "delete" key to remove selected visual elements.

4.3 Visualization Modification

Modification is defined as the response of the visualization after users perform actions. In the field of visualization interaction, the modifications have been classified in different ways [13, 34, 59]. Building upon these approaches, we summarized visualization modifications on a single existing chart based on the extent of changes made to the elements. Our classification considers factors including whether the elements' positions are changed, whether elements are added or removed, and whether the current encoding method is maintained. As depicted in Figure 3, we categorized visualization operations into six types.

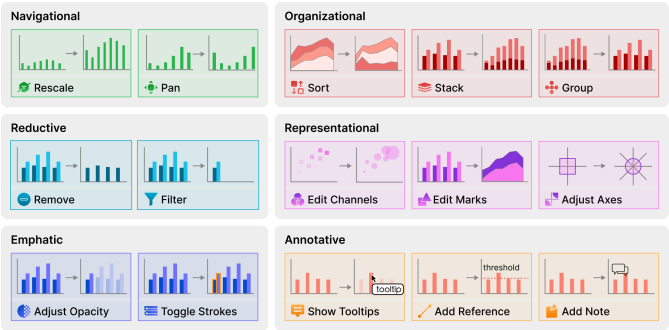


Figure 3: The design space of visualization modifications.

- **Emphatic** modification involves assigning varying visual intensities (e.g., opacity) to existing elements, distinguishing between user-focused and non-focused content. In our scenario, we have two kinds of emphatic modifications, namely, opacity manipulation and stroke emphasis. Opacity manipulation lowers the opacity of unfocused elements to emphasize focused ones. Border emphasis adds outline to focused elements. This modification relates to the "select" operation in both Yi et al. [59] and Munzner's [34] classifications.
- **Reductive** modification selectively retains focused elements while eliminating unfocused ones. Similar to highlighting in its intent to prioritize user-focused content, the reduce operation goes further by completely removing non-essential elements. We support two kinds of reductive modification, direct removing unfocused marks and removing according to filter condition. Reductive modification results in a simplified visualization that concentrates on focused data. Reductive modification corresponds to "filter" in both Yi et al. [59] and Munzner's [34] classifications.
- **Annotative** modification adds elements to current visualizations to give explanations on some visual marks. We support three types of annotative modifications: (1) tooltips (e.g., appear on hover), (2) reference lines (e.g., highlight thresholds or aggregates), and (3) text labels that directly annotate data points. This modification corresponds to "annotate" in Munzner's [34] classification.
- **Navigational** modification alters the view for visual marks. The modification in our approach includes two main operations: scaling and panning. Scaling changes the magnification level of the visualization (e.g., zooming into a selected area of a scatterplot). Panning adjusts which portion of the visualization is visible in the viewport. Both operations can be implemented by changing the domain and range of the axes. This modification does not alter the data itself but reframes the view, ensuring that the displayed content is appropriately scaled and positioned. Navigational modification belongs to "reconfigure" in Yi et al.'s [59] framework and corresponds to "navigate" in Munzner's [34] classification.
- **Organizational** modification involves altering the arrangement of elements within a visualization. Reorganization techniques include sorting, stacking, overlapping, and grouping. For instance, stacking can aggregate data into a summarized form (e.g., transforming grouped bars into stacked bars), while sorting elements along an axis can enhance ranking or comparison tasks. The modification belongs to "reconfigure" in Yi et al.'s [59] framework and "rearrange" in Munzner's [34] classification.
- **Representational** modification changes the visual encoding of a chart. We consider three types: (1) changing visual channels (e.g., color, size, or shape); (2) changing mark types (e.g., converting a line chart to an area or bar chart); and (3) changing axes (e.g., switching from a linear to a logarithmic scale). Changing the representation type is essential when the current encoding cannot support a required analysis. For example, in a multi-line chart of product sales trends, if the user wants the overall trend, re-encoding transforms it into a stacked area chart, where each mark represents a product and the total height shows the sum. *Representational* corresponds to "encode" in Yi et al. [59] and "change" in Munzner [34].

5 Multi-Agent Requirement Analyzer

The goal of the multi-agent requirement analyzer is to parse the user's requirements into interaction specifications (actions and modifications as defined in section 4).

5.1 Requirements

Parsing users' natural language instructions presents three main challenges: **Interpretation:** Users often lack technical terminology and describe the same interaction in varied ways (e.g., "mouseover" vs. "hovering"). The system must map such diverse expressions to precise specifications. **Execution:** Even well-interpreted instructions may yield technically infeasible specifications. The system must validate and refine results to ensure executability. **Specificity:** Users frequently give ambiguous and under-specified instructions (e.g., "make this chart interactive") [44]. The system must detect missing details and guide users toward complete specifications.

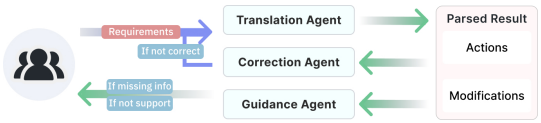


Figure 4: The process of LLM parsing users natural language input.

To address these challenges, we introduce a multi-agent architecture (see Figure 4), consisting of a *translation agent*, a *correction agent*, and a *guidance agent*. This multi-agent analyzer provides a robust system for translating natural language requirements into formal interaction specifications. The agents collaboratively guide users in authoring interactions, highlight errors in their instructions, and refine LLM-generated results, ensuring the final design meets explicit needs, addresses implicit expectations, and aligns with visualization interaction principles.

5.2 Translation Agent

This agent transforms user instructions into formal interaction specifications. Users often express their needs in natural language that can be ambiguous or underspecified. The translation agent interprets these instructions and maps them to our action-modification interaction model in section 4, ensuring that user intent is accurately reflected in the formal representation. The agent takes the user requirements, supported actions, and modifications as selectable inputs and outputs a structured interaction specification with determined user actions and modifications. Each action and modification has a target. When the actions happen, the modifications will be triggered. For example, the

translation agent processes the instruction *"Create a button that, when clicked, changes the chart to a bar chart."* as follows. The term *"click"* is directly mapped to the action click. The action target is an extra widget, a button. The modification "change" is mapped to representational modification, and the parameter is "bar chart". The system will create a button, when users click the button, the system will reencode the visualization into a bar chart.

5.3 Correction Agent

The correction agent acts as a validator, refining the translation agent's output to ensure accuracy and alignment with the user's intent. The correction agent reviews the output from the translation agent, identifying inconsistencies or technical infeasibilities that may arise from misinterpretations or limitations of multimodal large language models (MLLMs), and provides feedback to the translation agent. The correction agent checks the following:

- Check if the parsed results follow the action-modification structure required. The output should be a list of actions and modifications. For each action and modification, it verifies whether the target is compatible with it.
- Check if the users' requirements are satisfied using the output action and modifications. It confirms whether the translation agent's output accurately reflects the user's instructions.
- Check if the output modifications can be supported in the visualization: For example, stacking or grouping is not supported in a line chart, and the correction agent validates this.
- Check if additional modifications are needed: For instance, after removing some visual marks, the correction agent assesses whether rescaling the axes is necessary to optimize space usage.

By cross-referencing the translation agent's output with the design space and the user's original instructions, the correction agent enhances the system's reliability, making it more accessible to non-expert users. If discrepancies are detected, the correction agent provides feedback and requests the translation agent to regenerate the parsed results.

5.4 Guidance Agent

The agent identifies gaps in user requirements, highlights errors or ambiguities, and suggests complementary interactions. Because users often have incomplete mental models of possible or useful interactions, the guidance agent refines specifications by recommending contextually relevant interactions and steering users toward clearer, more effective requests. **Missing required information**. If users' instructions do not provide enough information about actions and modifications, the guidance agent will point out the missing information and give some feedback. If the possible action, modification have a default choice that is most commonly chosen, the guidance agent would produce the possible choice and ask to confirm. For example, a user inputs "show tooltip". Since no action and action target are explicitly mentioned, the guidance agent assumes the most common action, mouse over a visual element. The guidance agent assigns *visual mark* as the target of the action and *hovering* as the action. **Out of range.** When users' requirements are out of the current support range, the guidance agent will give feedback that points out the current action or modifications that are not supported and ask users to provide other requirements.

6 Visualization Abstraction Translator

This section presents our approach to abstracting visualizations into an implementation-agnostic representation: we first discuss the rationale for this representation (subsection 6.1), then introduce an MLLM-based parser for converting visualizations (subsection 6.2), and finally describe how the model supports visualization modifications (subsection 6.3).

6.1 Consideration for Representation

We require an implementation-agnostic representation for static visualizations that supports both cross-platform adaptability and subsequent modifications. The core requirements are to preserve the appearance of existing visualizations while enabling modification. Existing approaches range from high-level specifications such as Vega-Lite [40] to low-level implementations such as D3 [3]. Low-level representations precisely preserve appearance by directly controlling visual elements (for example, shapes, lines, and positions), but they lack structured abstractions that connect elements to data semantics, which makes modification difficult. High-level specifications, in contrast, encode abstract structures such as categories, axes, and data bindings. They are often used as intermediate representations in prior work [11, 12]. However, when high-level specifications are used to recreate an existing visualization, they regenerate all marks from scratch rather than reusing the original ones. As a result, the new marks no longer correspond directly to the originals, making it difficult to maintain visual consistency during subsequent interactions or modifications. Neither purely high-level specifications nor low-level implementations fully meet our needs. We therefore seek an intermediate representation that captures the appearance of existing visualizations while maintaining the flexibility required for modification.

Thus, we use a constraint-based representation [27] that abstracts visualizations modeled by control points and constraints. This representation meets our needs by **preserving appearance** while **enabling smooth, automatic modifications** through constraints. It fully maintains the appearance of original visual marks (e.g., shapes, colors, and sizes) and uses control points to represent different visual marks, such as employing contour lines to characterize polygons and paths. It expresses data mappings and spatial dependencies through various types of spatial constraint relationships. The constraints can drive visual marks to finish modifications fluently and automatically. These spatial constraints can be transformed into forces that guide the positions and positional changes of control points, allowing modifications to existing visualizations. For example, in a stacked bar chart where each bar represents a category, if one category is removed, the remaining bars automatically shift downward to fill the gap, guided by spatial constraints. In the constraint model, the **visual marks** (points, lines, areas, and text) are presented by **control points** and their connections, as depicted in Figure 5. The constraint model formulates the spatial positions of visual marks using three types of relationships.

- **Intra-mark relationships.** The constraint among control points within a visual mark defines the intrinsic shape and size. For instance, the width and height of a bar can be determined by adding *fixed constraints* in the corners (Figure 5 (a) and (b)).
- **Inter-mark relationships.** The constraints can describe the relative positions among visual marks. For example, the area visual mark in a stacked bar chart (Figure 5 (a)) has vertical *collision constraints*, which force a visual mark on top of another.
- **Mark-to-external relationships.** Constraints further express the relationships between a visual mark and external references. For example, an area mark in a stacked area chart is aligned with the x-axis, where the visual mark is driven by the *support constraint* and *gravity constraint* from the x-axis (Figure 5 (c)).

6.2 MLLM-based Constraint Parser

We propose an algorithm that leverages an MLLM to parse existing visualizations into a constraint-based representation. The parser operates in three stages: (1) extracting visual objects from the chart, (2) identifying the chart type and the roles of these objects, and (3) constructing constraints based on the identified structure.

6.2.1 Element Extraction

We employ JavaScript-based parsing scripts executed in a headless browser to analyze SVG files and extract visual elements. These elements are converted into a simplified vector format (SimVec [23]) for efficient processing. We identify SVG elements such as `<rect>`, `<text>`, and `<path>`. During this process, all CSS classes and inline styles are resolved into their final rendered properties, such as fill color, opacity, stroke width, and stroke color, while redundant style metadata and class identifiers are discarded. To eliminate the complexity of nested transformations, all coordinates are converted into absolute values. We then convert the data into the SimVec format. This preserves essential chart information while reducing token usage

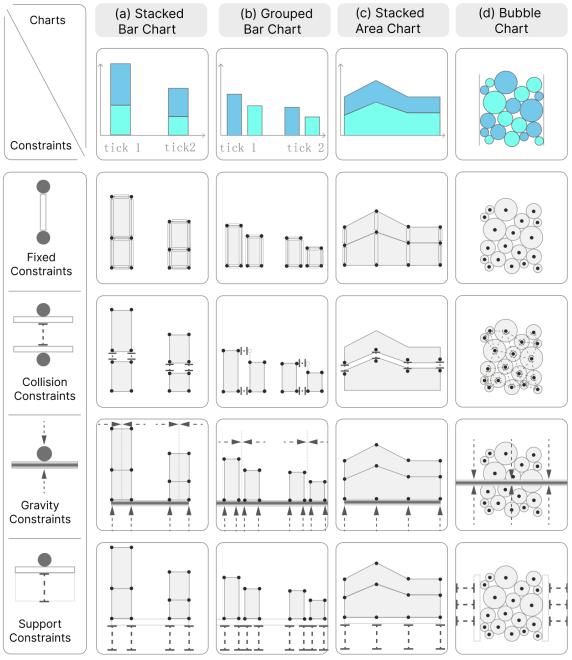


Figure 5: The constraint model [27] is employed as the representation.

by up to 90%, in practice. For instance, any closed path or geometric shape (e.g., path, rect, or circle) is abstracted into a structured format. A representative example of a polygon in SimVec format is: polygon: points$[(x_1,y_1),(x_2,y_2),\ldots]$, color: (h,s,l).

6.2.2 Element Identification

We employ an MLLM to classify the role of elements in a chart, for example, the axis tick, the title, and the legend. The SimVec data, combined with the corresponding bitmap representation of the SVG, are provided as inputs to the large language model.

- **Element-level identification**: We employ an MLLM to classify the role of the elements, which is also the fundamental step of previous reverse engineering work for visualization [37, 41]. Each element in a chart may be a data-encoded element or reference component. The data-encoded elements are those visual marks that directly encode data, e.g., a bar in a bar chart or a circle in a scatter plot. The reference elements are those elements that provide extra information for the data elements, for example, the legend or the title of a chart.
- **Chart-level identification**: The chart-level attributes are inferred by the MLLM, e.g., the chart type, axes type, and whether there are stacking/grouping/overlapping relationships.

6.2.3 Constraint Representation

After classifying the roles of each element, we can represent the visual marks using control points and constraints. With identified reference elements (legends, axes) and data-encoded elements (e.g., bars, points), we establish the complete data mappings for the input chart. We determine the precise spatial coordinates of each data element and their corresponding data values by calculating the axes scales and legend mappings based on the element classifications. This allows us to recover both the visual properties (position, size, color) and the corresponding data values for each mark in the visualization, effectively reconstructing the data-to-visual encoding relationships.

We organize constraints into steps (see Figure 5). We add *fixed constraints* to visual marks. In bar charts, we establish x and y fixed constraints between vertices of connected edges. For area marks, we apply fixed constraints between upper and lower points sharing the same x-coordinate. We apply *collision constraints* between control points of stacked elements. In stacked area charts, collision constraints prevent overlap between stacked marks. We apply *support constraints* to points of marks stacked on axes, ensuring they remain above the x-axis. *Gravity constraints* have two roles: marks stacked on axes experience forces toward the axes, ensuring alignment; in directions without stacking, constraints pull marks toward data value ticks. For example, in line charts, each point is pulled toward its data value. Gravity constraints ensure that elements respond to axis scale transformations.

6.3 Constraint-Based Modification

We translate modifications in subsection 4.3 into constraint changes.

- **Emphatic** modification doesn't alter the position or structure of the control points, and no constraint adjustments are needed. Emphatic modification is achieved by changing the opacity or stroke of selected and unselected visual marks.
- **Reductive** modification removes some visual marks from the visualization. When a mark is removed, the constraints automatically drive the positions of the remaining visual marks. For example, in a stacked area chart, when the bottom visual mark is removed, the upper ones automatically drop down.
- **Annotative** modification does not affect the spatial layout of visual marks, no constraint changes are needed. We directly add tooltips, lines, and text labels on top of the visualization.
- **Navigational** modification (i.e., rescaling and panning) involves changing the axis scale (e.g., zooming in on a bar chart). In the constraint model, constraints of control points are tied to the relative axes. When the scale of the axis changes, the related fixed, gravity, and support constraints are updated.
- **Structural** modification includes sorting, stacking, overlapping, and grouping. For sorting on axes, the control points representing these marks are governed by constraints tied to the axis (e.g., order bars by height). For stacking, grouping, and sorting of stacking, the order or direction of collision constraints is altered. For example, changing from a stacked bar chart to a grouped bar chart changes the collision constraint order from horizontal to vertical.
- **Representational** modification changes the visual mark types. For visual channel change and representation type change, we calculate new control points from existing ones to preserve spatial consistency. Constraints among new control points are given accordingly. For example, when we change a line chart to a bar chart, each point in the line chart is replaced by a new visual mark with four control points and fixed constraints among these points. For axes change, we change the coordinate type or scale of axes and update related constraints accordingly.

7 Interface

As shown in Figure 6, the interface has two views: a dialogue and a chart view. The creator can upload an SVG in the chart view, and specify their authoring requirements in the dialogue view.

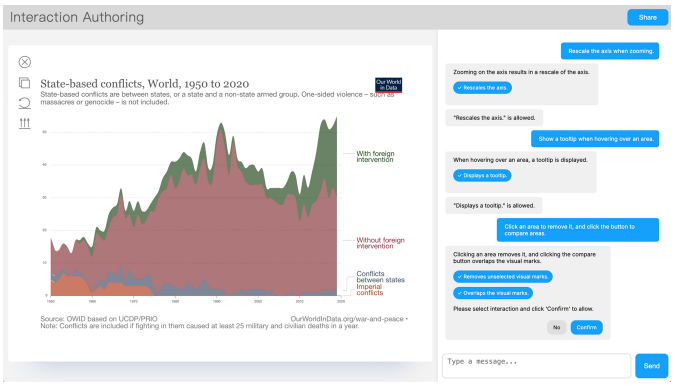


Figure 6: The interface of the visualization interaction authoring system. On the left is the chart view; on the right is the dialogue view.

Dialogue view. The dialogue view enables creators to author visualization interactions using natural language. Users can specify visualization interactions in natural language, and the multi-agent analyzer (section 5) translates these into structured interaction specifications. The system also provides guidance through the dialogue view when cre-

ators provide under-specified or out-of-range instructions. The dialogue process consists of three stages:

- **Message**: Users enter authoring instructions in natural language through the interactive interface. For example, a user might type, *"I want to rescale the x-axis when I zoom on the axis."*
- **Confirm**: After the user inputs an instruction, the analyzer converts it into a structured action-modification specification. The system then displays an NL explanation of this specification, detailing the actions and modifications to be applied to the uploaded visualization. Creators can refine the modifications by deselecting unwanted options. To proceed, the creator confirms the action-modification specification by clicking the "Confirm" button in the response.
- **Apply**: Once the user confirms the added interaction, the system applies the corresponding actions and modifications to the visualization in the chart view.

Chart view. Once a visualization is uploaded, it is immediately converted into a constraint-based representation. This representation retains the visual appearance of the uploaded visualization but alters its underlying implementation by replacing the visual marks with control points and their connections. After the creator adds and confirms actions and modifications in the dialogue view, these interactions are applied in the chart view. The actions are directly translated into inputs for corresponding hardware devices, such as mouse and keyboard inputs, and are associated with event listeners for the targeted elements. If the targets of the actions are additional widgets, the system creates these new widgets within the chart view. Simultaneously, the modifications are converted into changes to control points and constraints as described in subsection 6.3.

8 Evaluation

We evaluate the usability and effectiveness of the method through two case studies (subsection 8.1) and a user interview (subsection 8.2). We focus our evaluation on case studies and user interviews, since to our knowledge no existing systems support adding interactivity to pre-existing visualizations without coding.

8.1 Case Study

We present two cases to demonstrate how the proposed system interprets various types of natural language inputs and generates corresponding interactive modifications. To clearly describe the scenarios involving the creation and use of interactions, we use *creator* to refer to the person authoring the interaction and *user* to refer to the person engaging with the interactive visualization.

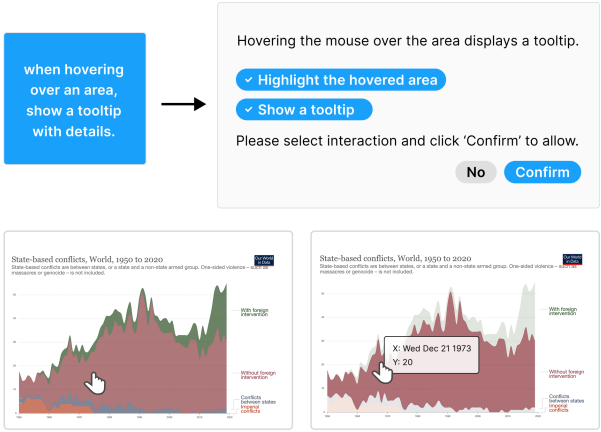


Figure 7: A creator defines an interaction: when hovering over a visual mark, a tooltip displays its details.

8.1.1 Case 1: Augmented Area Chart

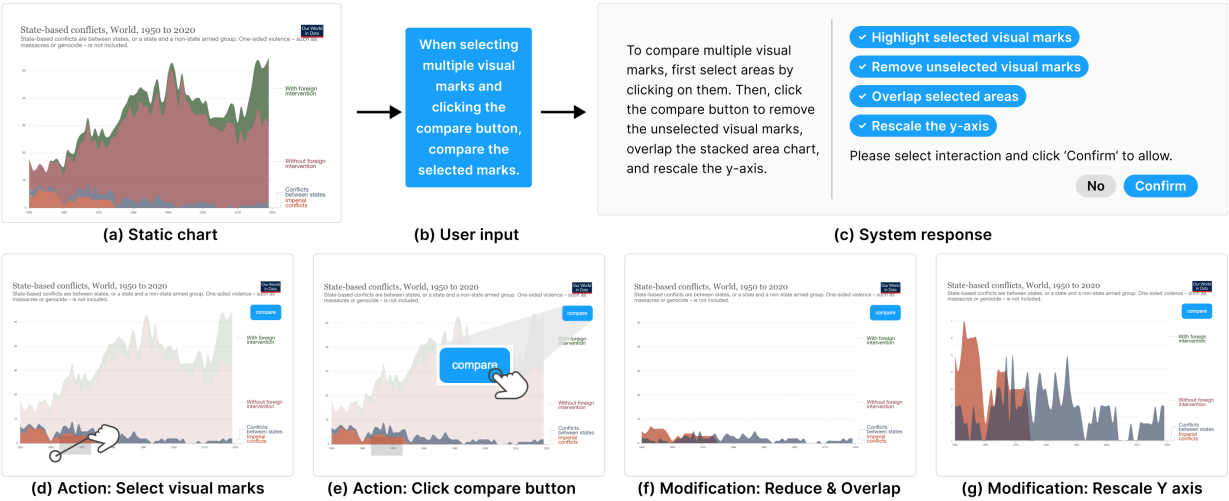
The original visualization is a stacked area chart. As shown in Figure 7, a creator provides the natural language input: *"when hovering over an area, show a tooltip with details."* The system processes this input into a structured interaction specification: `{action = "hover", action_target = "visual mark", modification = "annotate", modification_target = "visual mark"}`. As depicted in Figure 7, the possible modifications derived from this specification are displayed in the dialogue view for the creator to confirm. After adding interaction, the creator can deselect any unwanted modifications and click the confirm button. Once confirmed, the interaction is applied to the existing chart in the chart view to create an interactive one. After applying the interaction, when a user hovers over a visual mark (i.e., an area in this chart), the system shows a tooltip with more details about that mark. Figure 8 shows a more complex case with four modifications. The creator authors a more complex interaction: "When selecting multiple visual marks and clicking the compare button, compare the selected marks." The system parses this input into interaction specifications with multiple actions and modifications. The actions include selecting marks and clicking an *extra widget* (the compare button generated by the system). The system suggests several modifications (Figure 8 (c)), including: "highlight selected visual marks," "remove unselected visual marks," "overlap selected areas (for comparison)," and "rescale y-axis (for better space usage)". These parsed modifications are presented as selectable boxes, and once confirmed by the creator, the system applies the corresponding actions and modifications.

After enhancing the static visualization shown in Figure 8 (a) with interactive functionalities, the resulting visualization better supports users in the comparison task. As illustrated in Figure 8 (d), users can select two visual marks using a brush action. These marks occupy only a small portion of the chart area, making their corresponding values hard to discern in the original visualization. Moreover, because the marks are stacked, comparing them is challenging due to distortions, such as the sine illusion effect [5, 52], where stacked areas visually exaggerate or obscure differences in magnitude. The added interaction addresses this issue by aligning the selected visual marks, mitigating such perceptual distortions. After selection, users click the "Compare" button, introduced in Figure 8 (e). In response, the system removes unselected marks, separates the stacked relationship between the selected marks (Figure 8 (f)), and rescales the y-axis to optimize space usage (Figure 8 (g)). As a result, the selected marks are aligned and enlarged along the y-axis, enabling users to compare them effectively.

8.1.2 Case 2: Exploratory emissions chart

We use a multi-line chart from Our World in Data [36] that depicts global sulfur dioxide (SO_2) emissions by sector (industry, energy, and transport) from 1750 to 2022. Our system enables creators to augment such visualizations with interactions that support interactive data exploration. For example, if a creator specifies a button for switching to an area chart, the system automatically generates the corresponding control. Similarly, a button can be defined to switch to a bar chart. Creators can also author interactions that allow users to drag elements within the chart to transform overlapping areas into stacked areas. When a creator requests zooming on the x-axis, the system enables in/out zoom functionality to adjust the visible time range. Additionally, creators can define tooltips that reveal detailed data values when users hover over visual marks. Through this authoring process, the uploaded chart is transformed into an interactive visualization.

By combining different interactive functionalities, users can explore data from multiple perspectives. As shown in Figure 1 (a), the initial view employs a line chart to illustrate emission trends from various categories over time. When users need to understand the total contribution of each emission source, they can switch to an area chart by clicking the "change to area chart" button, transforming to Figure 1 (b). To see the total trend, users can drag a visual mark up to change the overlapped areas into stacked areas (Figure 1 (c)). Users can zoom into a specific period (e.g., 1974 - 1978) by adjusting the x-axis range (Figure 1 (d)). However, this view may not reveal precise values. Therefore, users can transform the visualization into a bar chart by clicking the "change to bar chart" button (Figure 1 (e)). As demonstrated in Figure 1 (f), the system supports detailed data inspection. When users hover over specific data points, the system displays an information tooltip containing precise values. For instance, the figure shows that SO_2



(a) Static chart (b) User input (c) System response

(d) Action: Select visual marks (e) Action: Click compare button (f) Modification: Reduce & Overlap (g) Modification: Rescale Y axis

Figure 8: A creator defines a comparison interaction with two actions and four sequential modifications. After deployment, when users select multiple visual marks and click the "compare" button, the interactive visualization transitions from (d) to (g) to support comparison.

emissions from international shipping were 50.3 units in 1977. This feature addresses limitations of visual estimation, enabling quantitative analysis, particularly in cases where multiple data series overlap or values are closely proximate.

8.2 User Interview

We evaluated *Athanor* through interviews with 11 participants experienced in data visualization and design.

8.2.1 Participants and Apparatus

To evaluate the system's usability and generalizability across users with different backgrounds, we recruited 11 participants (P1-P11, 5 males, 6 females) from various academic and professional fields, including mathematics, computer science, international trade, business engineering, and software development. This cohort included two visualization experts (**P2, P6**), two data analysts (**P3, P7**), one developer (**P9**), five postgraduate students (**P1, P4, P5, P8, P11**), and one undergraduate student (**P10**). All participants reported prior experience in creating or analyzing visualizations as part of their academic or professional work. Specifically, two participants (**P3, P6**) had 3-5 years of experience, five (**P2, P4, P8, P9, P11**) had 1-3 years, and four (**P1, P5, P7, P10**) had less than one year of experience. Our system was hosted on a server, allowing participants to access and interact with it using desktops.

8.2.2 Procedures

At the beginning of each session, we introduced the background and explained our goal of transforming static visualizations into interactive ones using natural language commands. To assess the comprehensiveness of our design space, participants first completed a pre-task questionnaire, where they listed the actions, modifications, and natural language commands they would like to use to add interactions. Next, we conducted a 10-minute demonstration showcasing the system's core features, including how users could specify interactions through natural language and how the system would apply these interactions to static visualizations. Following the demonstration, participants were encouraged to freely explore the system for 10 minutes to become familiar with its interface and functionality. Then, in the task phase, participants were asked to add interactive features to static visualizations described in subsection 8.1. We encouraged them to think aloud during the process, explaining the interactions they chose and their rationale for how these interactions would improve the visualization. Participants had 30 minutes to complete this task. Finally, participants completed a post-task questionnaire (Figure 9) including eight closed-ended questions (Q1-Q8) on system effectiveness, efficiency, and usability (rated on a 5-point Likert scale [15]), and three open-ended questions (Q9-Q11) on useful features, improvements, and suggestions.

8.2.3 Results

Figure 9 presents the results from both the pre-task and post-task questionnaires, and the detailed analysis is as follows.

Design Space Comprehensiveness. We analyzed the number (n) of actions and modifications mentioned by participants in the pre-task questionnaire and mapped them to our design space, as illustrated in the horizontal bar charts in Figure 9. Participants proposed a wide range of actions, such as *Click* (n=8), *Drag & Drop* (n=4), and *Zoom* (n=3), as well as corresponding modifications, including *Emphatic* (n=8), *Organizational* (n=7), *Reductive* (n=6), and *Annotative* (n=4). These results indicate that our system supports most user-intended actions and modifications. A few exceptions were noted, such as an *action* for adding interactive components (e.g., checkboxes) and a *modification* of triggering context menus, which fall outside the current scope of our design space. Additionally, the structure of natural language commands collected revealed a strong preference for full-form expressions in the pattern of *"Action + Target + Modification"* (n=39), indicating that users naturally specify interaction intents with all three components. Some commands only contained "*Target*" and "*Modification*" (n=12), often relying on implicit or default actions. For example, in the command "show tooltips on the points," the implied action is hovering or clicking. *Athanor* can still support such vague commands through the aforementioned guidance agent (subsection 5.4).

Effectiveness and Efficiency. All questions related to effectiveness and efficiency (Q1-Q5) received mean scores above 4, indicating that most participants found the system's functionality satisfactory. In particular, participants agreed that the system enabled them to create meaningful interactive visualizations aligned with their analytical goals (Q1) and that the natural language interface helped specify interactions (Q2). One exception was participant P1, who gave a neutral rating for Q1. She noted the need for a toggle button to switch between two interactions that shared the same action, which the current system did not support. Participants also generally found the system responsive and informative in interpreting their inputs (Q3). However, P6 pointed out that the textual feedback could be overwhelming when it was too long, and suggested that incorporating visual cues could make it easier to understand, especially when their intended interactions were not supported. All participants agreed that the system helped them save time when creating interactions (Q5), and most of them found the interaction creation process easy to use (Q4). The two neutral scores for Q4 were given by P11, who suggested adding auto-complete functions, and P6, who required a preview feature before applying changes.

Usability. Most participants agreed that they could quickly grasp how to use the system (Q6), indicating that the natural language interface and interaction framework were highly learnable. To further evaluate learning difficulty, we recorded the time each participant took to add their first interaction. Five participants (P2, P3, P6-P8) com-

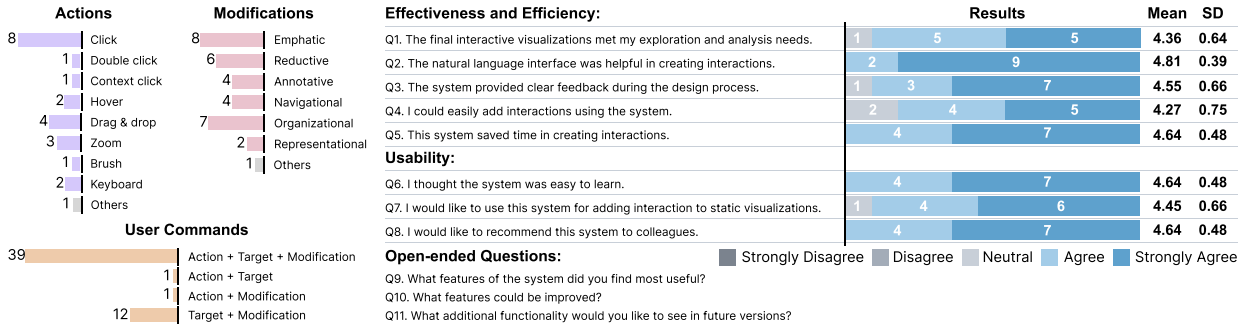


Figure 9: User interview results. Horizontal bar charts on the left show the actions, modifications, and potential commands mentioned by users during the pre-task questionnaire, indicating that our approach can cover most user needs for adding interactions to static charts. The questions in the post-task questionnaire and their results are shown on the right.

pleted the task within 5 minutes, another five (P1, P4, P5, P9, P10) within 5-10 minutes, and only one participant (P11) took more than 10 minutes. Participants also expressed a strong interest in continued use of the system (Q7) and would recommend it to others (Q8).

Open-ended Questions. In response to Q9, all participants highlighted the natural language interface for adding interactions as the most useful feature of the system. Even the two visualization experts (P2 and P5) emphasized that while coding offered greater flexibility, the natural language interface was more intuitive and easier to use for creating most types of interactions. Several participants also highlighted the specific strengths of the system. P9 commented, "*When my description is not clear enough, the system provides similar suggestions, which makes it easier for me to find the interaction I need.*" P4 appreciated the responsiveness, stating, "*The feedback is instant, so I can immediately tell whether the change is what I intended.*" P8 mentioned, "*Directly filtering out unwanted elements from the chart is incredibly convenient. Previously, I often had to rebuild the entire visualization to achieve the same result.*" In response to Q10, participants also suggested areas for improvement. P1 and P5 mentioned that the system's processing time for adding interactions could be faster. P11 suggested that the system should provide more examples at the beginning when the user is not familiar with the system. For future improvements (Q11), P3 and P9 expressed a desire for customized chart types like composite charts. P6 and P11 suggested adding voice command functionality to support hands-free interaction. P8 proposed the inclusion of a customizable color palette to enable more flexible styling of visualization elements.

9 Discussion and Future Work

Our approach transforms static visualizations into interactive ones through natural language requirements. This approach enabled interactivity in many previously static visualizations. However, several important considerations and limitations warrant discussion.

Scope of Interaction Support. The current approach derives data exclusively from the existing visualization. Consequently, the supported interaction modes are limited to manipulating information already present in the visualization through rearrangement. However, many interactions that require external data, such as drill-down operations where users seek more detailed information about elements of interest, exceed the capabilities of our current approach. When users request information beyond what can be inferred from the visual representation, the approach cannot respond adequately. Future work necessitates the integration of external data binding mechanisms to support these types of interactions.

Scalability and Adaptive Parsing. Our framework decouples the interaction logic synthesis from the visual rendering scale. For visualizations with high-cardinality elements, such as area charts containing thousands of data points, the multi-agent requirement analyzer focuses on structural patterns rather than individual visual mark, ensuring that logical complexity does not increase with data density. Currently, thousands of data points represent a typical upper bound for maintaining performance in web-based interactive visualizations, which remains within the context window limits of state-of-the-art MLLMs. To further ensure robustness, our system employs an adaptive parsing strategy: if the element count exceeds a safe threshold, the system can pivot to extracting only essential metadata and text elements (e.g., axis labels, legends, and titles). This allows the MLLM to infer the chart's semantic structure (axes and legends) and intent without being overwhelmed by excessive tokens.

Generalizability and Chart Types. Currently, *Athanor* primarily supports common statistical visualizations within the Cartesian coordinate system. Bar charts (simple, grouped, stacked), line charts, area charts (simple, overlapped, stacked), and scatter plots, are generally regarded as the most frequently used charts [1], which are all well-supported by our MLLM-based parsing and interaction mapping. However, our approach faces challenges with visualizations that require complex spatial layout computations or specialized data structures. For instance, force-directed graphs, geographic maps, and volumetric visualizations often involve non-linear spatial mappings or intricate topological relationships that are difficult to reconstruct purely from static SVG elements without underlying data models. Future work could involve enhancing the multi-agent reasoning capabilities to better infer these complex layouts or integrating lightweight geometry solvers to support a broader range of non-Cartesian visualizations.

Input Formats and Core Contributions. Our current implementation primarily operates on SVG-based visualizations, as our core contribution lies in the downstream interaction-authoring pipeline rather than the initial parsing of visual structures. SVG serves as a structured intermediate representation that enables the system to focus on the mapping from visual elements to interactive behaviors. Extending the system to support bitmap visualizations is an independent and decoupled process, which could utilize advanced MLLM-based techniques to extract visual hierarchies. For instance, recent work like SimVecVis [23] demonstrates how LLMs can effectively reconstruct visual elements and spatial structures from bitmap images. Since this parsing stage is decoupled from our core interaction-generation logic, future work could integrate such modules to make our framework format-agnostic.

10 Conclusion

We present *Athanor* for adding interactive capabilities to static visualizations based on natural language specifications. Our action-modification interaction design space provides a structured framework, abstracting interactions into user actions and corresponding visualization modifications. A multi-agent requirement analyzer translates natural language instructions into actionable specifications, accommodating diverse user needs without technical expertise. An implementation-agnostic representation translator converts static visualizations into flexible representations that can be manipulated independently of their original implementation. Evaluation results show that our approach effectively adds interactive functionalities to common chart types while preserving high visual fidelity to the original visualizations.

Acknowledgments

This project is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 2 (Proposal ID: T2EP202220049), and by the Start Up Grant awarded to Yong Wang. Any opinions, findings and conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

[1] L. Battle, P. Duan, Z. Miranda, D. Mukusheva, R. Chang, and M. Stonebraker. Beagle: Automated extraction and interpretation of visualizations from the web. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, pp. 1–8, 2018. 9

[2] D. Baur, B. Lee, and S. Carpendale. TouchWave: Kinetic multi-touch manipulation for hierarchical stacked graphs. In *Proceedings of the ACM Conference on Interactive Tabletops and Surfaces*, 10 pages, p. 255264, 2012. 2

[3] M. Bostock, V. Ogievetsky, and J. Heer. D^3: Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 9 pages, 2011. 2, 5

[4] M. Brehmer and T. Munzner. A multi-level typology of abstract visualization tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, 2013. 2

[5] C. Bu, Q. Zhang, Q. Wang, J. Zhang, M. Sedlmair, O. Deussen, and Y. Wang. Sinestream: Improving the readability of streamgraphs by minimizing sine illusion effects. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1634–1643, 2021. 7

[6] S. K. Card, J. Mackinlay, and B. Shneiderman. *Readings in information visualization: using vision to think*. Morgan Kaufmann, 1999. 3

[7] J. Choi, D. G. Park, Y. L. Wong, E. Fisher, and N. Elmqvist. VisDock: A toolkit for cross-cutting interactions in visualization. *IEEE Transactions on Visualization and Computer Graphics*, 21(9):1087–1100, 2015. 3

[8] K. Cox, R. E. Grinter, S. L. Hibino, L. J. Jagadeesan, and D. Mantilla. A multi-modal natural language interface to an information visualization environment. *International Journal of Speech Technology*, 4(3-4):297–314, 2001. 2

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL*, vol. 1, pp. 4171–4186, 2019. 2

[10] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios. DataTone: Managing ambiguity in natural language interfaces for data visualization. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, pp. 489–500, 2015. 2

[11] J. Harper and M. Agrawala. Deconstructing and restyling D3 visualizations. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, pp. 253–262, 2014. 3, 5

[12] J. Harper and M. Agrawala. Converting basic D3 charts into reusable style templates. *IEEE Transactions on Visualization and Computer Graphics*, 24(3):1274–1286, 2018. 3, 5

[13] J. Heer and B. Shneiderman. Interactive dynamics for visual analysis: A taxonomy of tools that support the fluent and flexible use of visualizations. *Queue*, 10(2):30–55, 2012. 1, 3, 4

[14] E. Hoque, V. Setlur, M. Tory, and I. Dykeman. Applying pragmatics principles for interaction with visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):309–318, 2017. 2

[15] A. Joshi, S. Kale, S. Chandel, and D. K. Pal. Likert scale: Explored and explained. *British journal of applied science & technology*, 7(4):396, 2015. 8

[16] S. E. Kahou, V. Michalski, A. Atkinson, Á. Kádár, A. Trischler, and Y. Bengio. FigureQA: An annotated figure dataset for visual reasoning. *arXiv preprint arXiv:1710.07300*, 2017. 2

[17] E. Kavaz, A. Puig, and I. Rodríguez. Chatbot-based natural language interfaces for data visualisation: A scoping review. *Applied Sciences*, 13(12):7025, 2023. 2

[18] D. H. Kim, E. Hoque, and M. Agrawala. Answering questions about charts and generating visual explanations. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 13 pages, p. 113, 2020. 2

[19] T. Kim, B. Saket, A. Endert, and B. MacIntyre. Visar: Bringing interactivity to static data visualizations through augmented reality. *arXiv preprint arXiv:1708.01377*, 2017. 3

[20] C. Lai, Z. Lin, R. Jiang, Y. Han, C. Liu, and X. Yuan. Automatic annotation synchronizing with textual description for visualization. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 2020. 2

[21] D. Li, H. Mei, Y. Shen, S. Su, W. Zhang, J. Wang, M. Zu, and W. Chen. ECharts: A declarative framework for rapid construction of web-based visualization. *Visual Informatics*, 2(2):136–146, 2018. 2

[22] G. Li, X. Wang, G. Aodeng, S. Zheng, Y. Zhang, C. Ou, S. Wang, and C. H. Liu. Visualization generation with large language models: An evaluation. *arXiv preprint arXiv:2401.11255*, 2024. 2

[23] C. Liu, C. Da, X. Long, Y. Yang, Y. Zhang, and Y. Wang. SimVecVis: A dataset for enhancing mllms in visualization understanding. In *Proceedings of the IEEE Visualization and Visual Analytics*, pp. 26–30, 2025. 5, 9

[24] C. Liu, Y. Guo, and X. Yuan. AutoTitle: An interactive title generator for visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 30(8):5276–5288, 2024. 2

[25] C. Liu, Y. Han, R. Jiang, and X. Yuan. ADVISor: Automatic visualization answer for natural-language question on tabular data. In *Proceedings of the IEEE Pacific Visualization Symposium*, pp. 6–15, 2021. 2

[26] C. Liu, L. Xie, Y. Han, and X. Yuan. AutoCaption: An approach to generate natural language description from visualization automatically. In *Proceedings of the IEEE Pacific Visualization Symposium*, pp. 191–195, 2020. 2

[27] C. Liu, Y. Zhang, C. Wu, C. Li, and X. Yuan. A spatial constraint model for manipulating static visualizations. *ACM Transactions on Interactive Intelligent Systems*, 14(2):1–29, 2024. 2, 3, 5, 6

[28] M. Lu, J. Liang, Y. Zhang, G. Li, S. Chen, Z. Li, and X. Yuan. Interaction+: Interaction enhancement for web-based visualizations. In *Proceedings of the IEEE Pacific Visualization Symposium*, pp. 61–70, 2017. 3

[29] Y. Luo, X. Qin, N. Tang, and G. Li. Deepeye: Towards automatic data visualization. In *Proceedings of International Conference on Data Engineering*, pp. 101–112, 2018. 2

[30] Y. Luo, N. Tang, G. Li, C. Chai, W. Li, and X. Qin. Synthesizing natural language to visualization (nl2vis) benchmarks from nl2sql benchmarks. In *Proceedings of the ACM SIGMOD*, 13 pages, p. 12351247, 2021. 2

[31] Y. Luo, N. Tang, G. Li, J. Tang, C. Chai, and X. Qin. Natural language to visualization by neural machine translation. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):217–226, 2021. 2

[32] J. Ma and M. Agrawala. Mover: Motion verification for motion graphics animations. *ACM Transactions on Graphics*, 44(4):Article 33, Aug. 2025. 3

[33] P. Maddigan and T. Susnjak. Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models. *Ieee Access*, 11:45181–45193, 2023. 2

[34] T. Munzner. *Visualization Analysis and Design*. 2014. 2, 4

[35] OpenAI. ChatGPT API. `https://beta.openai.com/docs/api-reference/introduction`, 2023. Accessed: April 1, 2025. 2

[36] Our World in Data. Sulfur dioxide emissions by sector, world. `https://ourworldindata.org/explorers/air-pollution?country=~OWID_WRL&Pollutant=Sulfur+dioxide&Sector=Breakdown+by+sector`, 2025. Accessed: 2025-04-07. 7

[37] J. Poco and J. Heer. Reverse-engineering visualizations: Recovering visual encodings from chart images. *Computer Graphics Forum*, 36(3):353–363, 2017. 6

[38] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140):1–67, 2020. 2

[39] R. Sadana, M. Agnihotri, and J. T. Stasko. Touching data: A discoverability-based evaluation of a visualization interface for tablet computers. *CoRR*, abs/1806.06084, 2018. 2

[40] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. 2, 3, 5

[41] M. Savva, N. Kong, A. Chhajta, F.-F. Li, M. Agrawala, and J. Heer. ReVision: Automated classification, analysis and redesign of chart images. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, pp. 393–402, 2011. 6

[42] K. Sedig and P. Parsons. Interaction design for complex cognitive activities with visual representations: A pattern-based approach. *AIS Transactions on Human-Computer Interaction*, 5(2):84–133, 2013. 2

[43] V. Setlur, S. E. Battersby, M. Tory, R. Gossweiler, and A. X. Chang. Eviza: A natural language interface for visual analysis. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, pp. 365–377, 2016. 2

[44] V. Setlur and M. Tory. How do you converse with an analytical chatbot? revisiting gricean maxims for designing analytical conversational behavior. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2022. 4

[45] L. Shen, E. Shen, Y. Luo, X. Yang, X. Hu, X. Zhang, Z. Tai, and J. Wang. Towards natural language interfaces for data visualization: A survey. *IEEE*

Transactions on Visualization and Computer Graphics, 29(6):3121–3144, 2022. 2

[46] D. Shi, Y. Guo, M. Guo, Y. Wu, Q. Chen, and N. Cao. Talk2Data: High-level question decomposition for data-oriented question and answering. *CoRR*, abs/2107.14420, 2021. 2

[47] B. Shneiderman. The eyes have it: a task by data type taxonomy for information visualizations. In *Proceedings of IEEE Symposium on Visual Languages*, pp. 336–343, 1996. 2

[48] L. S. Snyder and J. Heer. DIVI: Dynamically interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):403–413, 2023. 1, 3

[49] C. Stolte, D. Tang, and P. Hanrahan. Polaris: A system for query, analysis, and visualization of multidimensional relational databases. *IEEE Transactions on Visualization and Computer Graphics*, 8(1):52–65, 2002. 2

[50] Y. Sun, J. Leigh, A. Johnson, and S. Lee. Articulate: A semi-automated model for translating natural language queries into meaningful visualizations. In *Proceedings of SG*, pp. 184–195, 2010. 2

[51] P. Vaithilingam, E. L. Glassman, J. P. Inala, and C. Wang. Dynavis: Dynamically synthesized ui widgets for visualization editing, 2024. 2

[52] S. VanderPlas and H. Hofmann. Signs of the sine illusion - Why we need to care. *Journal of Computational and Graphical Statistics*, 24(4):1170–1190, 2015. 7

[53] C. Wang, J. Thompson, and B. Lee. Data formulator: Ai-powered concept-driven visualization authoring. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):1128–1138, 2023. 2

[54] Y. Wang, Z. Hou, L. Shen, T. Wu, J. Wang, H. Huang, H. Zhang, and D. Zhang. Towards natural language-based visualization authoring. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):1222–1232, 2022. 2

[55] M. O. Ward, G. Grinstein, and D. Keim. *Interactive data visualization: foundations, techniques, and applications*. AK Peters/CRC Press, 2010. 2

[56] T. L. Weissgerber, V. D. Garovic, M. Savic, S. J. Winham, and N. M. Milic. From static to interactive: transforming data visualization to improve transparency. *PLoS biology*, 14(6):e1002484, 2016. 1

[57] T. Wolf, J. Chaumond, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, pp. 38–45, 2020. 2

[58] L. Xie, Y. Lin, C. Liu, H. Qu, and X. Shu. Datawink: Reusing and adapting svg-based visualization examples with large multimodal models.(2025). *arXiv preprint arXiv:2507.17734*, 2025. 3

[59] J.-S. Yi, Y. ah Kang, J. Stasko, and J. A. Jacko. Toward a deeper understanding of the role of interaction in information visualization. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1224–1231, 2007. 1, 2, 3, 4

[60] C. Zhu-Tian, W. Tong, Q. Wang, B. Bach, and H. Qu. Augmenting static visualizations with paparvis designer. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, pp. 1–12, 2020. 3